CH 2025-08-30

Standard IEEE-754.

Base binaria ($b=2$).

Viene utilizzata una rappresentazione con arrotondamento "round to even", ovvero l'ultima cifra della mantissa rappresentata dalla funzione fl è 0. Tuttavia, la maggiorazione dell'errore relativo di rappresentazione continua a valere.

Viene implementato un "gradual underflow". Pertanto, essendo la base binaria, la mantissa di un numero normalizzato sarà del tipo:

$1.f$, con f la parte frazionaria.

Similmente, la mantissa di un numero denormalizzato, sarà del tipo:

$0.f$, con f la parte frazionaria.

Questi argomenti ci dicono che, sapen-

do se il numero è normalizzato o meno, non abbiamo bisogno di memorizzare la sua parte intera, ma è sufficiente memorizzare la sola frazione f, risparmiando, quindi, 1 bit.

Lo standard prevede due tipi di numeri reali:

- singola precisione;
- doppia precisione.

Singola precisione.

Vengono allocati 4 byte (32 bit) così ripartiti:

1 per il segno della mantissa;

8 per le cifre dell'esponente ($s = 8$);

23 per la frazione f ($m = 24$);

Da questo segue che la precisione di macchina (singola precisione IEEE-754)

vale $u = \frac{1}{2} b^{1-m} = 2^{-24} \approx 10^{-7.5}$ (2 sopra b, 24 sopra m).

Vediamo riguardo all'esponente. Con 8 cifre binarie, si possono rappresentare tutti gli interi in $\{0, 1, \dots, 255\}$. Pertanto, $0 \le e \le 255$. In particolare, se:

$0 < e < 255$, allora il numero è **normalizzato**, e lo **shift** vale $\nu = 127$.

Se $e = f = 0$, allora abbiamo la rappresentazione dello ϕ.

Se $e = 0 \wedge f \neq 0$, allora il numero è **denormalizzato**, e lo **shift** vale $\nu = 126$.

OSS La variazione di shift, quando si denormalizza, si spiega osservando che il più **piccolo** numero normalizzato (positivo) è

$$1.0___0 \cdot 2^{1-127} = 2^{-126}.$$

(shift normalizzato)

Invece, il più **grande** numero **denormalizzato**

(positivo) è:

$$0.11\ldots1 \cdot 2^{-126}.$$

Pertanto, i due numeri sono "contigui".

Se $e = 255$ e $f = 0$, allora abbiamo

$+Inf$, se $s_0 = 0$,

$-Inf$, se $s_0 = 1$.

Se $e = 255$ e $f \neq 0$, allora abbiamo un NaN (Not a Number).

Questo è, ad esempio, originato da forme indeterminate del tipo

$Inf - Inf$

$0 * Inf$

Inf / Inf

$0/0$

Doppia precisione

In questo caso, vengono utilizzati 8 byte (64 bit) per rappresentare un numero in

doppia precisione. Questi sono così ripartiti:

1 bit per il segno della mantissa;

52 bit per la frazione f ($m = 53$);

11 bit per l'esponente e.

OSS In questo caso, la precisione di macchina vale $u = \frac{1}{2} b^{1-m} = \frac{1}{2} 2^{1-53} = 2^{-53}$

$\approx 10^{-16}$,

il che vuol dire che lavoriamo con circa 16 cifre decimali significative.

Da quanto esposto, $e \in \{0, 1, \dots, 2047\}$.

In modo sostanzialmente analogo al caso della singola precisione, si ha che:

$0 < e < 2047$, allora il numero è normalizzato, e lo shift vale $v = 1023$.

se $e = 0 \wedge f = 0$, allora abbiamo ϕ.

se $e = 0 \wedge f \neq 0$, allora il numero è denormalizzato, e lo

shift vale $\nu = 1022$.

$e = 2047 \wedge f = 0$, allora abbiamo:

$+Inf$, se $\alpha_0 = 0$,

$-Inf$, se $\alpha_0 = 1$.

$e = 2047 \wedge f \neq 0$, abbiamo un NaN.

Aritmetica Finita.

Se eseguiamo operazioni algebriche elementari $(+/-, *, /) \ni \oplus$. Allora

$\forall x, y \in \mathbb{R}$: $x \oplus y = fl(fl(x) \oplus fl(y))$

Questo implica che, di norma, le proprietà algebriche delle operazioni (associatività, distributività, ecc.) non valgono più.

```
>> y = 1e-308, x=y; for i = 1:51, x = x*2; end, for i = 1:51, x=x/2; end, [y x y-x]
y =
    9.999999999999999e-309
ans =
    9.999999999999999e-309    9.999999999999999e-309                        0
>> y = 1e-308, x=y; for i = 1:51, x = x/2; end, for i = 1:51, x=x*2; end, [y x y-x]
y =
    9.999999999999999e-309
ans =
    9.999999999999999e-309    1.112536929253601e-308   -1.125369292536008e-309
>>
```

```
>> y = 1e-308, x=y; for i = 1:52, x = x*2; end, for i = 1:52, x=x/2; end, [y x y-x]
y =
    9.999999999999999e-309
ans =
    9.999999999999999e-309    9.999999999999999e-309                        0
>> y = 1e-308, x=y; for i = 1:52, x = x/2; end, for i = 1:52, x=x*2; end, [y x y-x]
y =
    9.999999999999999e-309
ans =
    9.999999999999999e-309                        0    9.999999999999999e-309
>>
```

```
r2 =
    1.7976931348623l6e+308
>> [(r2-r2)*2 r2*2-r2*2]
ans =
     0   NaN
>> [(r2-r2)+1 r2-(r2+1)]
ans =
     1     0
>>
```

Conversione tra tipi diversi.

x variabile in doppia precisione;

y variabile in singola precisione.

PI doppia precisione π (3.14159265358...)

$$\begin{cases} x = PI \\ y = x \end{cases}$$

allora y conterrà π con la massima accuratezza consentita alla singola precisione.

Viceversa,

$$\begin{cases} y = PI \\ x = y \end{cases}$$

allora x conterrà π con l'accuratezza della singola precisione.

Inoltre,

$x = 1e308$, x, da $9.99... \cdot e308$

$y = 1e308$, y, da Inf

Talora è necessario convertire anche tra numeri di tipo reale e tipo intero.

La conversione intero → reale, in genere è innocua, a parte il fatto che in genere non si ha più un intero, questo perché il range di rappresentazione degli interi è più ristretto di quello del tipo reale ($\mathbb{I}$). Il viceversa non è vero, perché $\mathbb{I}$ è generalmente molto più ampio dell'insieme di rappresentabilità del tipo intero. Nel caso del filmato di Ariane V, il problema è stato originato dalla assegnazione di una variabile in doppia precisione, legata alla componente tangenziale della velocità, ad una variabile intera di 2 byte!!! Quindi, se il numero è maggiore di 32767, ho un errore (e che errore!).